

# Zalord — Hệ thống Chat Microservices

## Kiến trúc & Triển khai

Nhóm 17

Trường Đại học Công nghệ Thông tin — ĐHQG TP.HCM  
GVHD: TS. Nguyễn Duy Khánh

Tháng 6, 2026

# Nội dung trình bày

- 1 Tổng quan hệ thống
- 2 Domain-Driven Design
- 3 Nguyên lý thiết kế Microservices
- 4 Giao tiếp giữa các dịch vụ
- 5 Thiết kế & Quản lý cơ sở dữ liệu
- 6 Triển khai Docker & Kubernetes
- 7 API Gateway
- 8 Authentication & Authorization
- 9 Logging, Monitoring, Tracing
- 10 CI/CD & Triển khai
- 11 Resiliency Scenarios
- 12 Kết luận

# Giới thiệu Zalord

## Zalord là gì?

- Ứng dụng chat real-time kiến trúc **Microservices**
- **7 services** độc lập, polyglot Java / Go
- Hỗ trợ: nhắn tin 1-1, nhóm, file đính kèm, real-time presence
- Frontend: React 19 + TypeScript + Vite + WebSocket

## Tech stack tổng quan

- **Gateway:** Kong 3.7 (DB-less declarative)
- **Messaging:** RabbitMQ / Kafka (switchable)
- **Storage:** PostgreSQL ×6, Redis, MinIO (S3)
- **Observability:** Prometheus + Grafana
- **Deploy:** Docker Compose (local) / k3s VPS (dev) / GKE (prod)
- **CI/CD:** GitHub Actions → Docker Hub → GKE / k3s

# Danh sách Microservices

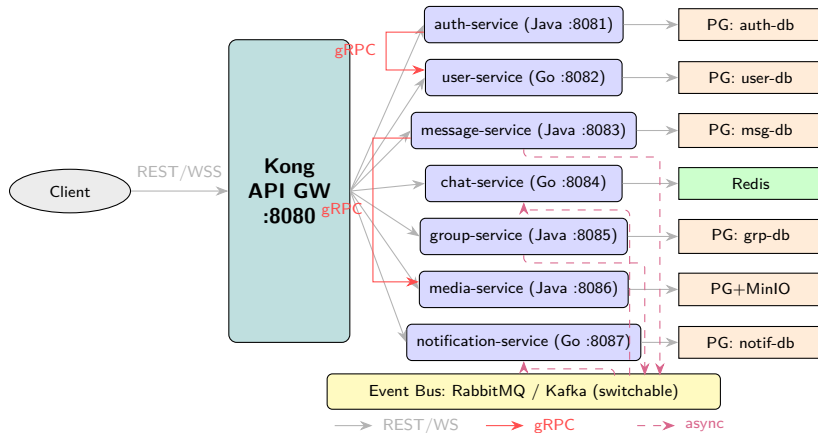
| Service              | Lang        | Port |
|----------------------|-------------|------|
| auth-service         | Java/Spring | 8081 |
| user-service         | Go          | 8082 |
| message-service      | Java/Spring | 8083 |
| chat-service         | Go          | 8084 |
| group-service        | Java/Spring | 8085 |
| media-service        | Java/Spring | 8086 |
| notification-service | Go          | 8087 |

## gRPC Internal (không qua Kong)

auth → user (CreateProfile :9082)

message → media (ValidateAttachments :9086)

# Kiến trúc tổng quan



# Tại sao không dùng Modulithic? (Req 1)

## Lý do chọn Microservices

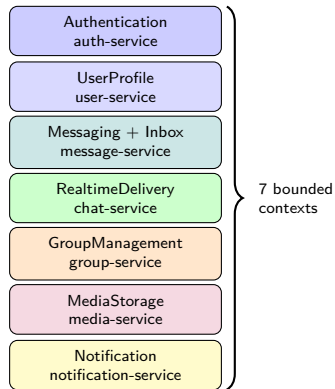
- **Polyglot language:** chat-service dùng Go (goroutines) cho WebSocket fan-out hiệu năng cao; business logic dùng Java Spring Boot
- **Independent scaling:** chat-service cần scale riêng khi traffic tăng, không kéo theo auth-service
- **Independent deploy:** cập nhật media-service không ảnh hưởng chat real-time
- **Fault isolation:** media-service lỗi không kéo sập toàn bộ hệ thống

## Nếu là Modulithic...

- Không thể mix Go + Java trong 1 process
- Scale toàn bộ khi chỉ cần scale 1 module
- Deploy toàn bộ khi chỉ thay đổi 1 module
- 1 memory leak / goroutine leak làm sập tất cả

# Phân chia theo DDD — Bounded Contexts (Req 2)

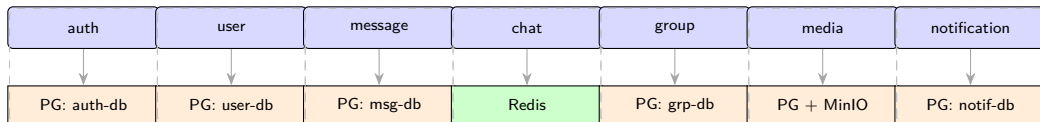
| Service              | Bounded Context   | Aggregate Root            |
|----------------------|-------------------|---------------------------|
| auth-service         | Authentication    | Account, Session          |
| user-service         | UserProfile       | UserProfile               |
| message-service      | Messaging + Inbox | Message, ConversationView |
| chat-service         | RealtimeDelivery  | Connection                |
| group-service        | GroupManagement   | Group                     |
| media-service        | MediaStorage      | MediaItem                 |
| notification-service | Notification      | Notification              |



# DB per Bounded Context (Req 2)

## Nguyên tắc phân chia

- Mỗi service có **1 bounded context** độc lập
- Không share entity/table giữa các service
- Giao tiếp qua **API / Event** (không gọi thẳng DB)
- conversation.id được tái sử dụng làm group.id (shared key, không shared schema)



Mỗi service chỉ truy cập database của chính mình — không share schema

Redis (shared): auth sessions, chat presence/pub-sub, message membership cache

MinIO (shared): avatars, attachments



# Single Responsibility Principle (Req 3)

## SRP — Mỗi service 1 trách nhiệm rõ ràng

- **chat-service**: CHỈ quản lý WebSocket connections + fan-out messages → không write business DB, không có business logic
- **media-service**: CHỈ quản lý upload lifecycle (generate presigned URL → MinIO → finalize metadata)
- **notification-service**: CHỈ consume events + persist + serve notification list
- **auth-service**: CHỈ authentication (issue/validate token, session management)

## Điểm tranh luận

message-service đảm nhận 2 vai trò: (1) write model cho messages, (2) project inbox read-model (CQRS). Có thể tách thành inbox-service nếu scale lên.

# High Cohesion & Low Coupling (Req 3)

## High Cohesion — Low Coupling

- **High Cohesion:** dữ liệu và logic cùng domain nằm trong 1 service, không scatter
- **Low Coupling:** chỉ **2 synchronous dependencies** (gRPC):
  - auth → user (registration)
  - message → media (validate attachments)
- Mọi giao tiếp còn lại: **async event bus**
- Downstream services không parse JWT — trust header từ Kong

## Execution Pipeline

Request → Kong (validate) → Service → DB / Event Bus

Không có cross-service synchronous chain dài hơn 2 hops

# Sync vs Async — Khi nào dùng gì? (Req 4)

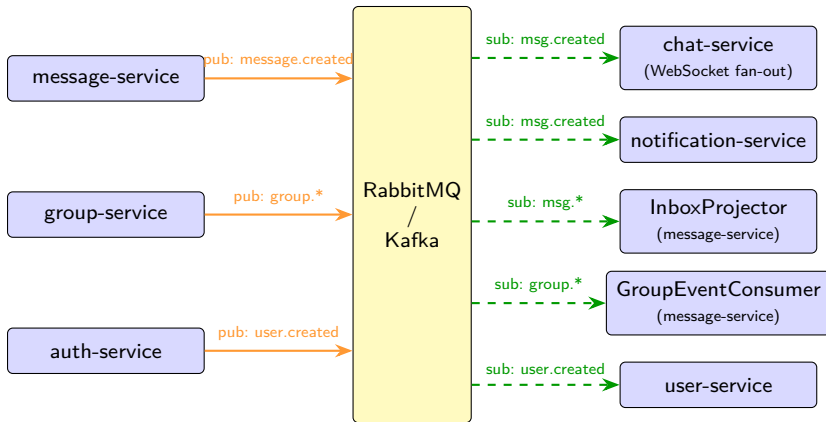
## Synchronous — gRPC (2 trường hợp)

- **auth** → **user-service** (CreateProfile):
  - Cần response ngay để rollback nếu profile creation fail
  - Deadline 5 giây, circuit breaker Resilience4j
- **message** → **media-service** (ValidateAttachments):
  - Validate media IDs trước khi persist message
  - Deadline 3 giây (hot path user-facing)
- Dùng gRPC vì: type-safe contract (Protobuf), binary encoding nhanh hơn REST, không qua Kong (internal network)

## Asynchronous — Event Bus (Kafka / RabbitMQ)

- **message.created** → chat-service (WebSocket fan-out), notification-service, InboxProjector
- **group.\*** (created/member-added/removed/updated) → message-service (project Conversation)
- **user.created** → user-service
- Dùng async vì: fan-out 1-to-many, producers không cần chờ consumers, failure isolation
- **Switchable**: EVENT\_BUS=rabbitmq hoặc kafka — cùng interface EventBus (Factory pattern)

# Event-Driven Architecture — Message Flow (Req 4)



# Thiết kế Endpoint & Swagger (Req 5)

## REST API — Naming Convention

- Base path: `/api/v1/{resource}` (plural noun)
- POST `/api/v1/auth/register`
- GET `/api/v1/users`
- GET `/api/v1/conversations`
- POST `/api/v1/messages`
- POST `/api/v1/groups/{id}/members`
- DELETE `/api/v1/groups/{id}/members/{uid}`
- POST `/api/v1/media/upload-url`
- WebSocket: `/ws/chat?token={jwt}`

## Error Response

Chuẩn **RFC 7807**  
(Platform-agnostic):  

```
{"type":...,  
  "status":400,  
  "detail":"...",  
  "title":"..."}
```

# Swagger / OpenAPI Aggregation (Req 5)

## Swagger / OpenAPI Aggregation

- Java services: **springdoc-openapi** → /v3/api-docs
- Go services: **swagger** → /v3/api-docs
- Kong route /api-docs/{svc} → `strip_path: true` → forward đến service /v3/api-docs
- **Swagger UI** container tại /docs aggregate tất cả

## [PLACEHOLDER — chụp hình]

*Chụp màn hình Swagger UI tại  
<http://localhost:8080/docs>  
hiển thị API docs từ tất cả 7 services*

# DB per Service & Consistency (Req 6)

## DB per Service

- 6 PostgreSQL instances độc lập + Redis + MinIO
- Không service nào truy cập trực tiếp DB của service khác
- **Strong consistency**: trong 1 service (ACID transactions)
- **Eventual consistency**: cross-service qua event bus
  - group member change → message-service projection có độ trễ nhỏ
  - message.created → inbox view update không đồng thời

## Consistency cần thiết ở đâu?

- **Strong**: auth (login/register), message persist (không mất tin nhắn)
- **Eventual**: inbox unread count, notification, presence status

# CAP Theorem & Saga Pattern (Req 6)

## CAP Theorem

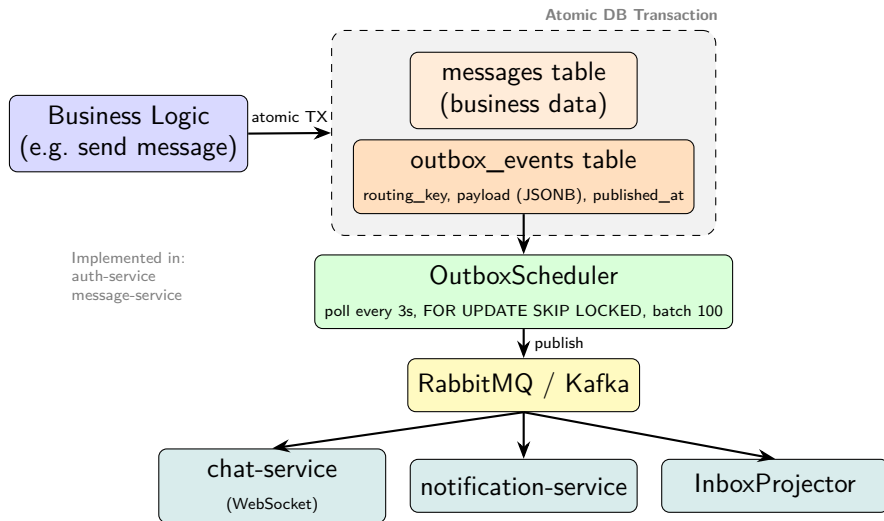
- PostgreSQL: **CP** (Consistency + Partition Tolerance)
- Chấp nhận availability giảm khi partition
- Redis: **AP** — eventual consistency cho presence data

## Vì sao không dùng Saga Pattern?

Message persist là local transaction (ghi vào 1 DB của message-service). Distributed transaction duy nhất là Registration (auth → user) — chỉ gồm 2 bước nên dùng gRPC + Circuit Breaker (Resilience4j) là đủ an toàn, không cần độ phức tạp của Saga.

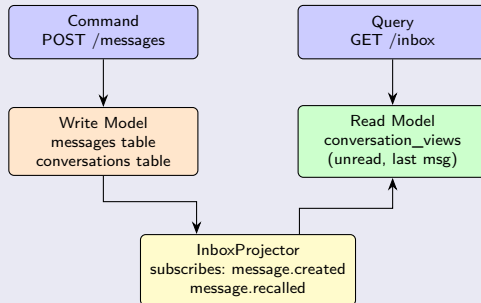


# Transactional Outbox Pattern (Req 7)



# CQRS trong message-service (Req 7)

## CQRS trong message-service



projection có độ trễ nhỏ (eventual consistency)

# Polyglot Persistence (Req 7)

## Polyglot Persistence

| Store         | Dùng cho                    |
|---------------|-----------------------------|
| PostgreSQL ×6 | Business data (ACID)        |
| Redis         | Sessions, presence, pub/sub |
| MinIO (S3)    | Avatars, attachments        |

[PLACEHOLDER]

*Polyglot: PostgreSQL +  
Redis + MinIO đang chạy  
trong Docker Compose*

# Dockerfile & Docker Compose (Req 8)

## Multi-Stage Dockerfile — Java (auth-service)

```
# Stage 1: Build
FROM eclipse-temurin:21-jdk-jammy AS build
WORKDIR /workspace
COPY mvnw . && COPY .mvn .mvn
COPY pom.xml .
RUN chmod +x mvnw && ./mvnw -B dependency:go-offline
# Layer cache: deps downloaded before source copy
COPY src src
RUN ./mvnw -B -DskipTests clean package \
    && java -Djarmode=layertools -jar target/*.jar \
        extract --destination target/extracted

# Stage 2: Runtime (JRE only, alpine = small)
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring
COPY --from=build /workspace/target/extracted/ ./
EXPOSE 8081
```

## Multi-Stage Dockerfile — Go (chat-service)

```
# Stage 1: Build
FROM golang:1.26.1-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download          # deps cached separately
COPY . .
RUN CGO_ENABLED=0 go build -o server ./cmd/server

# Stage 2: Runtime (scratch-like, ~8MB)
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/server .
EXPOSE 8084
CMD ["/server"]
```

# Docker Compose (Req 8)

## Docker Compose

- 20+ containers, 1 network zalord-net
- Named volumes per DB (pgdata\_auth, ..., miniodata, redisdata)
- Env vars từ .env file, không hardcode

## Kustomize Overlays

- Path: `deploy/k8s/overlays/dev/`
- Namespace: `zalord-dev`
- **Workloads:** 7 Deployments (services)
- **StatefulSets:** Kafka, RabbitMQ, Redis, PostgreSQL×6, MinIO (kèm Persistent Volumes 1–5Gi)
- **Resources:** req/limit (e.g. auth: 100m/384Mi req)
- **Probes:** TCP liveness, HTTP `/health`
- **Secrets:** SOPS + Age encryption

## [PLACEHOLDER — chụp hình]

```
kubectl get pods -n zalord-dev
```

*Chụp màn hình tất cả pods  
ở trạng thái Running*

## [PLACEHOLDER — chụp hình]

*Lens / K9s IDE hoặc*  

```
kubectl get all -n zalord-dev
```

  
*hiển thị cluster overview*

# Service Discovery (Req 9)

## Service Discovery — K8s DNS

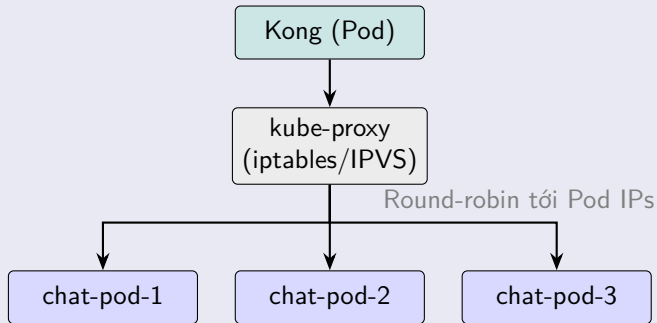
- K8s tự tạo DNS entry cho mỗi ClusterIP Service
- `auth-service.zalord-dev.svc.cluster.local`
- Các service gọi nhau qua tên service nội bộ (e.g. `http://user-service:8082`)
- **KHÔNG cần Eureka hay Consul** — K8s DNS thay thế hoàn toàn

## Cấu hình gRPC target

- `ZALORD_USER_SERVICE_GRPC_TARGET=user-service:9082`
- `ZALORD_MEDIA_SERVICE_GRPC_TARGET=media-service:9086`
- Quá trình: DNS resolve → ClusterIP → Pod IP

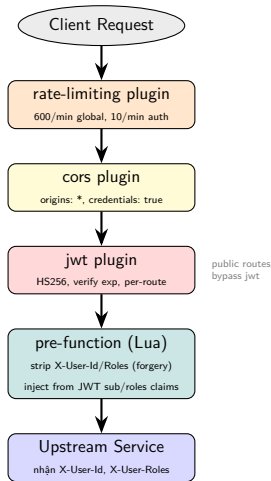
# Load Balancing (Req 9)

## Load Balancing — Server-side





# Kong API Gateway — Filter Chain (Req 10)



## Các tính năng Kong đã cấu hình

- **Rate Limiting**: 600 req/min per IP (global), 10 req/min (auth routes /login, /register)
- **CORS**: global, tất cả origins (dev), credentials enabled
- **JWT Filter**: validate HS256, kiểm tra exp claim
- **Identity Injection**: Lua pre-function inject X-User-Id (từ sub) và X-User-Roles (từ roles array)
- **Prometheus**: metrics tại admin :8001/metrics

# Kong — DB-less Mode & Route Table (Req 10)

## Declarative Config (DB-less)

- Config tại `infra/kong/kong.yml` (version control). **Cùng 1 file** cho Dev & Prod (tránh config drift).
- `__JWT_SECRET__` được thay bằng env var khi start — bảo mật secret.
- **KHÔNG dùng K8s Ingress Controller** (`networking.k8s.io/v1`). Đây là **lựa chọn thiết kế** standalone (xem `EXPLANATION.md`).

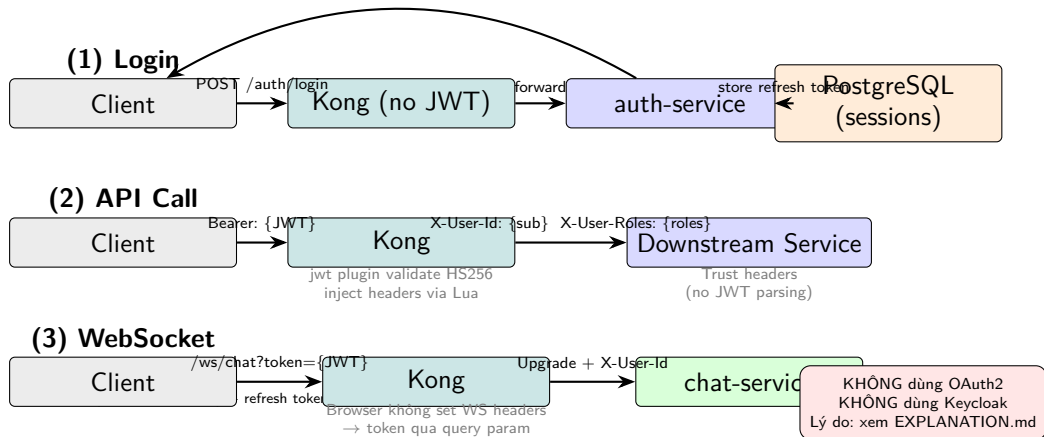
## Route Table

| Route                              | Upstream                           | JWT |
|------------------------------------|------------------------------------|-----|
| <code>/api/v1/auth/*</code>        | <code>auth:8081</code>             | ×   |
| <code>/api/v1/users</code>         | <code>user:8082</code>             | ✓   |
| <code>/api/v1/messages</code>      | <code>message:8083</code>          | ✓   |
| <code>/ws/chat</code>              | <code>chat:8084</code>             | ✓   |
| <code>/api/v1/groups</code>        | <code>group-service:8085</code>    | ✓   |
| <code>/api/v1/media</code>         | <code>media-service:8086</code>    | ✓   |
| <code>/api/v1/notifications</code> | <code>notification-svc:8087</code> | ✓   |
| <code>/docs</code>                 | <code>swagger-ui:8080</code>       | ×   |

## [PLACEHOLDER — chụp hình]

*Kong Admin API:*  
`curl localhost:8001/routes`  
  
*hoặc Kong Manager UI*  
*hiển thị danh sách routes/plugins*

# JWT Authentication Flow (Req 11)



# ConfigMap — Non-sensitive config (Req 12)

## ConfigMap — Non-sensitive config

- **Shared:** shared-config.yaml
  - EVENT\_BUS=rabbitmq
  - RABBITMQ\_HOST=rabbitmq
  - KAFKA\_BOOTSTRAP=kafka:9092
  - REDIS\_HOST=redis
  - JWT\_EXPIRY\_MINUTES=15
- **Per-service:** {svc}-config.yaml — database host/port/name, gRPC targets, service-specific config
- Mount as env vars vào Deployment

## Tại sao cần ConfigMap?

- Thay đổi config không cần build lại Docker image
- Khác nhau giữa environments (dev/staging/prod) qua Kustomize overlays

## Secret — Sensitive data (Req 12)

### Secret — Sensitive data (SOPS encrypted)

- **Shared:** DB passwords, JWT\_SECRET, MQ credentials
- **Per-service:** database URLs với credentials
- Files: \*.secret.enc.yaml — encrypted với **SOPS + Age key**
- Age key lưu trong **GitHub Actions Secret**
- CI/CD decrypt tại deploy time: `sops -d file.enc.yaml`
- Secret encrypted at rest trong Git — an toàn commit public repo

### Kong JWT Secret

kong.yml chứa `__JWT_SECRET__` placeholder. Container entrypoint dùng `sed` substitute từ K8s Secret env var. Secret không bao giờ nằm trong file config.

# Prometheus & Grafana — Observability (Req 13)

## Prometheus Scrape Config

| Target                      | Endpoint                                     |
|-----------------------------|----------------------------------------------|
| auth, message, group, media | /actuator/prometheus (Micrometer)            |
| user, chat, notification    | /metrics (client_golang + Gin)               |
| Kong                        | kong:8001/metrics (built-in plugin)          |
| RabbitMQ                    | rabbitmq:15692/metrics (rabbitmq_prometheus) |
| Kafka                       | kafka-exporter:9308 (sidecar)                |

## Metrics được track

- HTTP latency, request count, status codes (Micrometer, Kong)
- Active WebSocket connections (chat-service custom metric)
- Message throughput, queue lag (Kafka exporter)

## [PLACEHOLDER — chụp hình]

*Grafana dashboard*

`http://localhost:3000`

*Chụp màn hình panels:*

- HTTP request rate/latency
- Active WS connections
- Message throughput
- Pod resource usage

## [PLACEHOLDER — chụp hình]

*Prometheus targets page*

`localhost:9090/targets`

*all targets: State=UP*

# OpenTelemetry & Monitoring — Chưa triển khai (Req 13)

## OpenTelemetry — Chưa triển khai

### [PLACEHOLDER]

OpenTelemetry distributed tracing sẽ cho phép:

- Trace 1 request qua nhiều hops: Kong → message-service → media gRPC → outbox → event bus → chat
- Xác định latency ở từng hop
- Tích hợp với Jaeger / Tempo (Grafana)

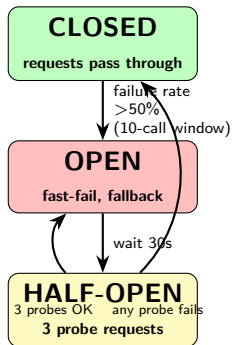
*[Diagram: trace flow từ Kong request đến consumer cuối cùng]*

## K8s Cluster Monitoring

- Dùng **Lens IDE** hoặc **k9s** CLI để giám sát cluster k3s

*[PLACEHOLDER: chụp Lens/k9s với zalord-dev namespace]*

# Circuit Breaker Diagram (Req 14)



Resilience4j 2.2.0



# Timeout, Circuit Breaker & Recovery (Req 14)

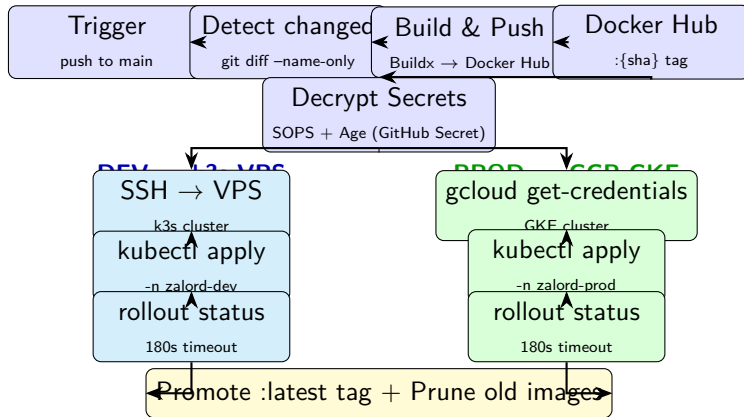
## Implementation

- **auth-service** → user-service gRPC:
  - `@CircuitBreaker(name="userGrpc")`
  - gRPC deadline: **5 giây**
  - Fallback: `createProfileFallback()`
- **message-service** → media-service gRPC:
  - `@CircuitBreaker(name="mediaGrpc")`
  - gRPC deadline: **3 giây** (hot path)
  - Fallback: `validateFallback()`

## Go services — Error Handling

- **PermanentError**: drop message (ack), log error
- **Transient error**: nack + requeue → RabbitMQ retry
- Frontend WebSocket: exponential backoff reconnect (cap 15s)

# GitHub Actions Pipeline (Req 15)



[PLACEHOLDER — chụp hình]

GitHub Actions → tab Actions → chọn workflow run thành công → chụp màn hình các steps

# Versioning & Rollback Strategy (Req 15)

## Image Versioning

- Mỗi image tagged với **Git SHA**:
  - `docker.io/{user}/zalord-auth:{sha}`
  - `docker.io/{user}/zalord-chat:{sha}`
  - ...
- Sau khi deploy thành công: promote `:latest` tag
- **Selective build**: chỉ rebuild service có file thay đổi → tiết kiệm thời gian CI
- Không có image nào bị overwrite — SHA tag immutable

## Testing trong CI — Chưa có

### [PLACEHOLDER]

CI/CD pipeline hiện tại: build + deploy. Chưa có bước automated testing.

Nên bổ sung:

- Unit tests (JUnit 5 cho Java, Go test cho Go)
- Integration tests (Testcontainers)
- Contract tests giữa services

### [PLACEHOLDER — chụp hình]

*Docker Hub repository  
hiển thị tags theo SHA:*

# Scenario 1: Circuit Breaker Activation (Req 16)

## Scenario: media-service xuống → CB open

- 1 Kill media-service pod:  
`kubectl delete pod media-service-xxx`
- 2 Gửi message có attachment qua API
- 3 Observe: **circuit breaker OPEN** sau >50% failures trong 10 calls (Resilience4j)
- 4 Response: fallback method trả về error (không hang)
- 5 Restart media-service pod (K8s tự restart)
- 6 Observe: CB → HALF-OPEN → 3 probe requests → CLOSED

## [PLACEHOLDER — chụp hình/log]

*Logs của message-service:*

- CB state: CLOSED → OPEN
- Fallback called
- CB state: OPEN → HALF-OPEN
- CB state: HALF-OPEN → CLOSED

*Hoặc Grafana metric panel cho Resilience4j state transitions*

## Kết quả kỳ vọng

- Hệ thống không crash khi media-service lỗi
- Fallback response trả về ngay (không chờ timeout)

## Scenario 2: Pod Crash & Self-Healing (Req 16)

### Scenario: chat-service pod crash

- 1 Client A và B đang chat real-time
- 2 `kubectl delete pod chat-service-xxx -n zalord-dev`
- 3 K8s phát hiện pod gone → tạo pod mới (Deployment replicas=1)
- 4 Frontend WebSocket: **exponential backoff reconnect** (cap 15s, logic trong `websocket.ts`)
- 5 Pod mới Ready → clients reconnect thành công

### [PLACEHOLDER — chụp hình]

```
kubectl get pods -n zalord-dev
```

*Chụp: chat-service pod  
RESTARTS count = 1, Running*

*Frontend console log:  
— WS disconnected  
— Reconnecting in 1s...  
— Reconnecting in 2s...  
— WS connected*

## Scenario 3: Outbox resiliency (Req 16)

### Scenario 3: Outbox resiliency

- ❶ Tắt RabbitMQ tạm thời
- ❷ Gửi messages → lưu vào outbox\_events table (atomic với message)
- ❸ Bật lại RabbitMQ
- ❹ **OutboxScheduler** (poll 3s) phát hiện unpublished events → publish
- ❺ Không mất message nào

### [PLACEHOLDER]

*Grafana: HTTP request rate  
dip khi pod đang restart,  
recover sau reconnect*

## Đã triển khai

- ✓ DB-per-service (6 PostgreSQL + Redis + MinIO)
- ✓ Transactional Outbox (auth, message services)
- ✓ CQRS (message-service InboxProjector)
- ✓ Event-driven (RabbitMQ/Kafka switchable)
- ✓ gRPC sync + Circuit Breaker (Resilience4j)
- ✓ Kong API Gateway (JWT, rate-limit, CORS, Lua)
- ✓ JWT HS256 auth (không cần

## Chưa triển khai / Không phù hợp

- **OpenTelemetry** distributed tracing (chưa implement)
- **Automated testing** trong CI (chưa implement)
- **ELK Stack** — resource constraint; stdout logging đủ cho scope này
- **Saga Pattern** — không có distributed transaction đủ phức tạp
- **OAuth2/Keycloak** — không phù hợp: closed first-party system

## Xem chi tiết

**Cảm ơn thầy và các bạn  
đã lắng nghe!**